



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 16 de abril del 2026

CALIFICACIÓN: _____

ACTIVIDADES REALIZADAS

1.- Helicóptero con dos luces spot light

Instrucciones: Agregar luz de tipo spotlight para el helicóptero de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight en el suelo hacia adelante y al retroceder (mover con teclado hacia dirección de X positiva) ilumine con un spotlight el suelo hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo a la dirección en la que esté moviéndose el helicóptero.

Window.h

Se agregaron los getters para el cambio de faros entre la spotlight que apuntaba hacia enfrente y la spotlight que apuntaba hacia atrás en el helicóptero

```
// --- VARIABLES PARA EL HELICÓPTERO ---
GLfloat getMovHelicoptero() { return movHelicoptero; } // Getter para el helicóptero
int getFaroHeliC() { return faroHeliActual; }           // Para cambio de faros
```

Window.cpp

Inicializamos el estado del faro en 7 por defecto, para que al arrancar el programa el helicóptero ya tenga una luz encendida. Además, en la función maneja teclado se colocó la tecla 5 y 6 para avanzar y retroceder, junto con una asignación del faro del helicóptero actual (índice 7 o 6)

```
movHelicoptero = 0.0f; //Heli
faroHeliActual = 7;    //Heli
...
// --- HELICÓPTERO ---
// [5 y 6]: Movimiento en Z y cambio direccional del faro (hacia adelante o atrás)
if (key == GLFW_KEY_5) {
    theWindow->movHelicoptero -= 0.5f;
    theWindow->faroHeliActual = 7;
}
if (key == GLFW_KEY_6) {
    theWindow->movHelicoptero += 0.5f;
    theWindow->faroHeliActual = 6;
}
}
```

Main

Dentro de nuestro int main, donde declarábamos el almacén de luces del ejercicio anterior agregamos nuestras dos spotlights amarillas. Finalmente, para el bucle por cada ciclo de reloj, modificamos la jerarquía del faro, calculando primero la posición global del helicóptero que se modifica con nuestro getter getMovHelicoptero. Spotlights en su índice 1 tomará el valor de faroHeliActual dependiendo de la tecla presionada.

```
// 6: HELICOPTERO FARO AMARILLO PERO HACIA ATRAS
farosAuxiliares[6] = SpotLight(1.0f, 1.0f, 0.0f,           // Color Amarillo (RGB)
    1.0f, 1.0f,
    0.0f, 0.0f, 0.0f,
    1.0f, -1.0f, 0.0f,
    1.0f, 0.045f, 0.0075f,
    30.0f);
// 7: HELICOPTERO FARO AMARILLO PERO HACIA ADELANTE
farosAuxiliares[7] = SpotLight(1.0f, 1.0f, 0.0f,           // Color Amarillo (RGB)
    1.0f, 1.0f,
    0.0f, 0.0f, 0.0f,
    -1.0f, -1.0f, 0.0f,
    1.0f, 0.045f, 0.0075f,
    30.0f);
...
...
```

```

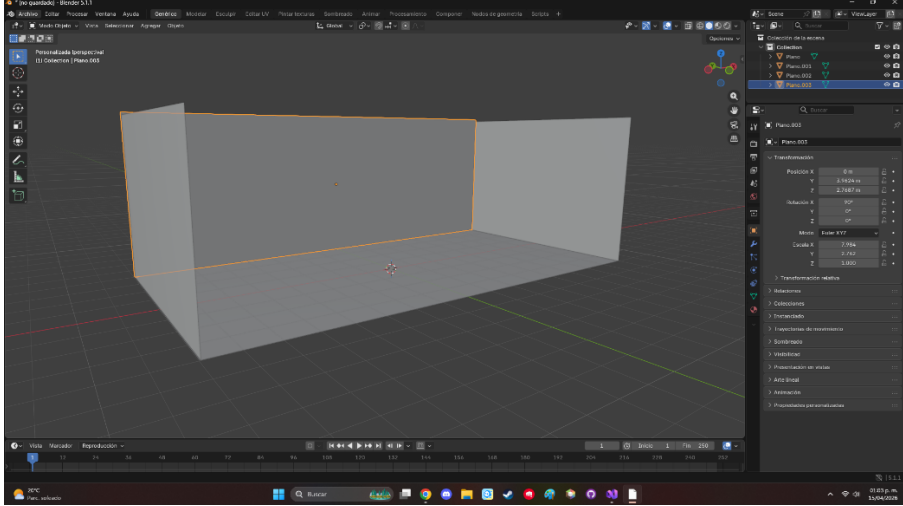
...
// JERARQUÍA DEL FARO AMARILLO (INDICE 1 en spotLights)
posHelicoptero = glm::vec3(0.0f + mainWindow.getMovHelicoptero(), 5.0f, 8.0f);
matrizHeliLuz = glm::mat4(1.0);
matrizHeliLuz = glm::translate(matrizHeliLuz, posHelicoptero);
posGlobalHeli = matrizHeliLuz * posLocalHeli;
spotLights[1] = farosAuxiliares[mainWindow.getFaroHeli()];
spotLights[1].SetPos(glm::vec3(posGlobalHeli));

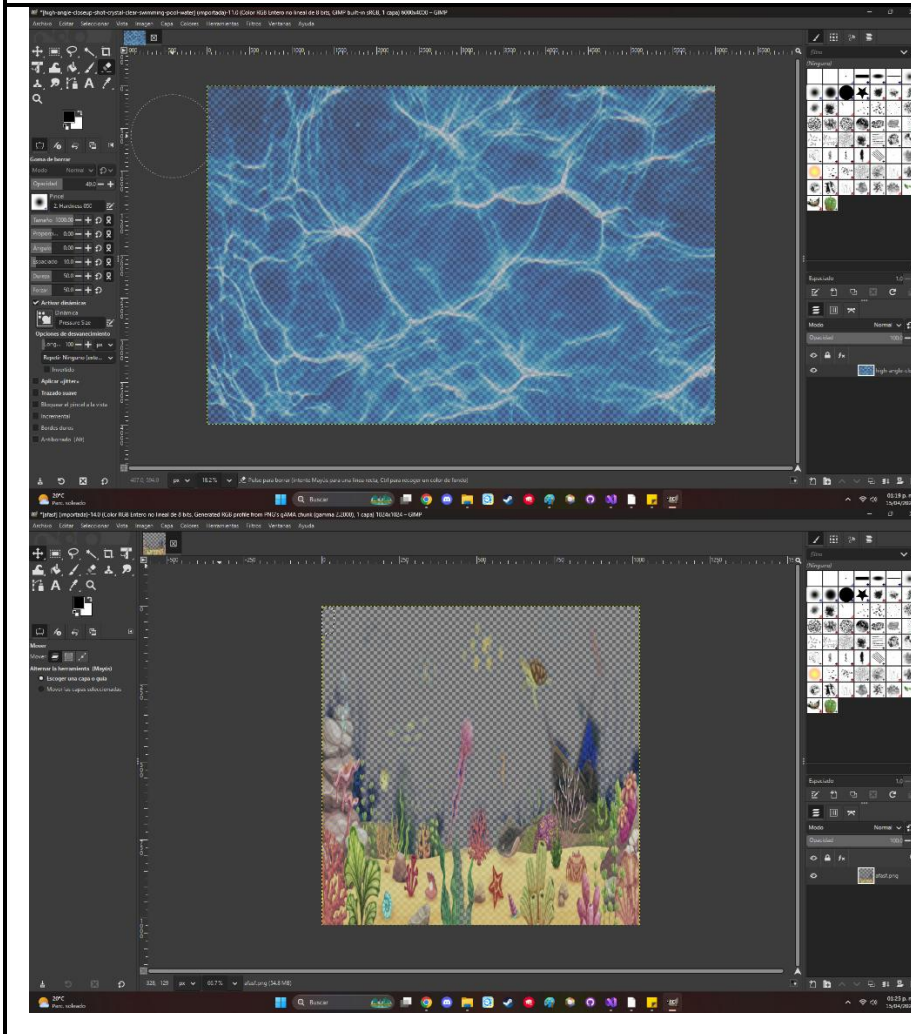
```

2.- Pecera y abisal

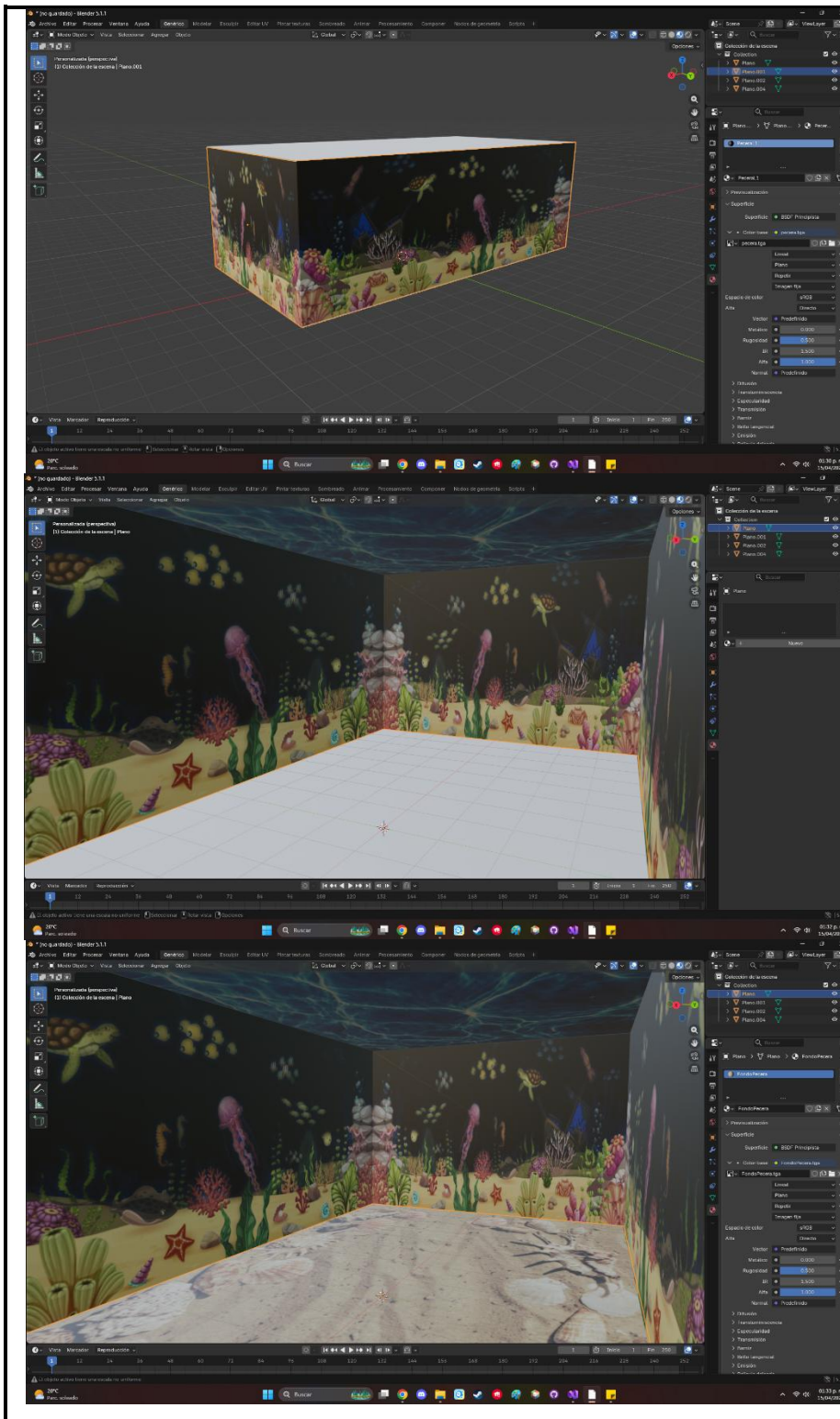
Instrucciones: *Crear o instanciar una pecera (traslucida) con las normales hacia adentro y en la parte superior imagen de agua e instanciar dentro de la pecera al pez abisal con movimiento de teclado para subir y bajar en diagonal.*

Para este ejercicio se modelo desde 0 una pecera en blender y se texturizo el pez dado por el profesor en el mismo software, a continuación, se presentan la serie de pasos realizados y pruebas, cabe señalar que las texturas fueron obtenidas de la página freepik.es

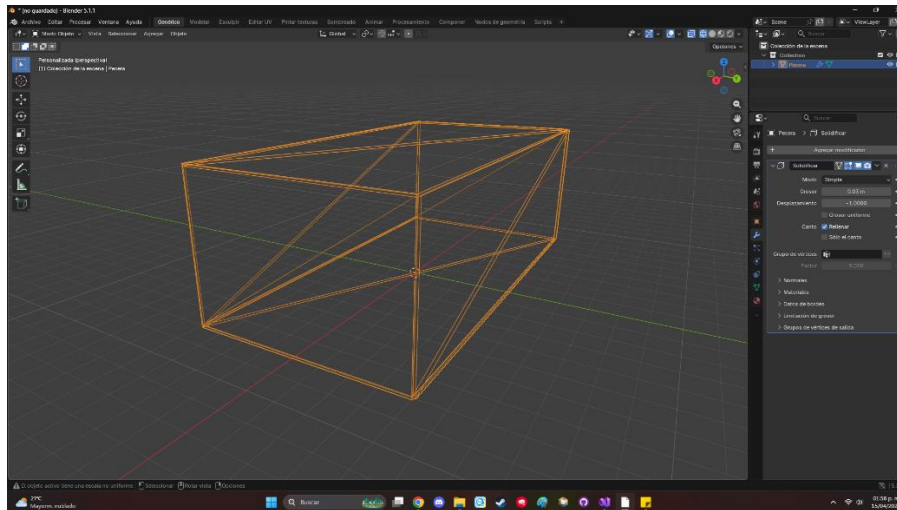
IMAGEN	DESCRIPCIÓN
	Se colocaron un total de 6 planos en blender, mismos que se escalonaron y trasladaron para que quedaran lo más parecido a una pecera.
	Se descargaron múltiples imágenes a utilizar como texturas de nuestra pecera.



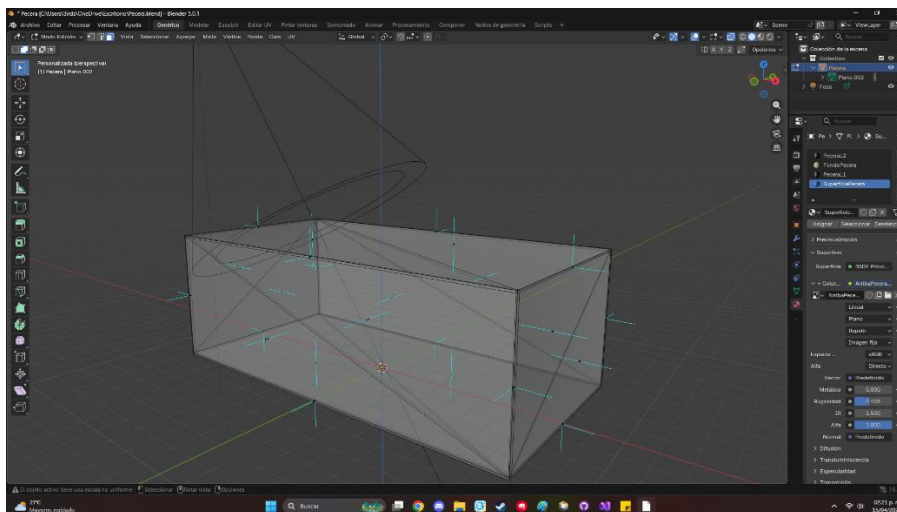
Estas imágenes se pasaron por el editor de imágenes GIMP para darles mejores filtros, así como para activar la transparencia de las mismas y exportarlas como archivos TGA



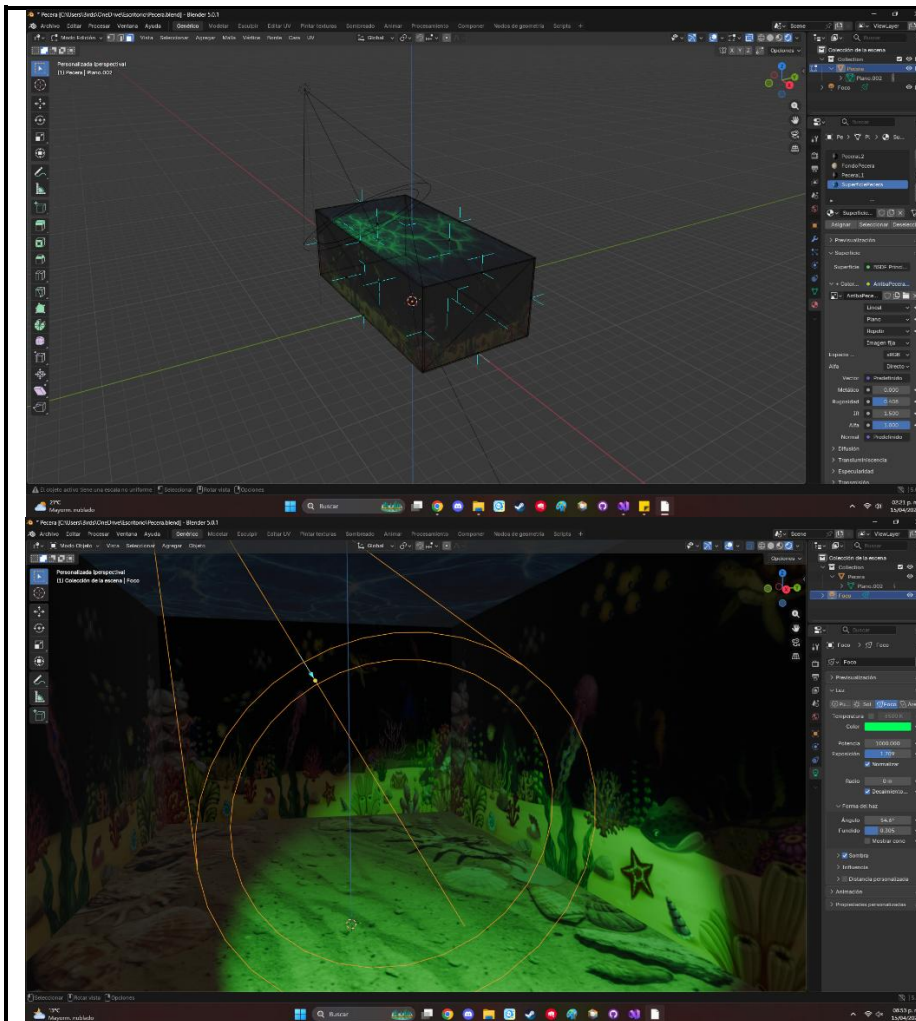
Regresando a blender, se usaron mapas UV para texturizar nuestra pecera. Es importante mencionar que después de este paso la pecera se unió en un solo objeto y se trianguló.



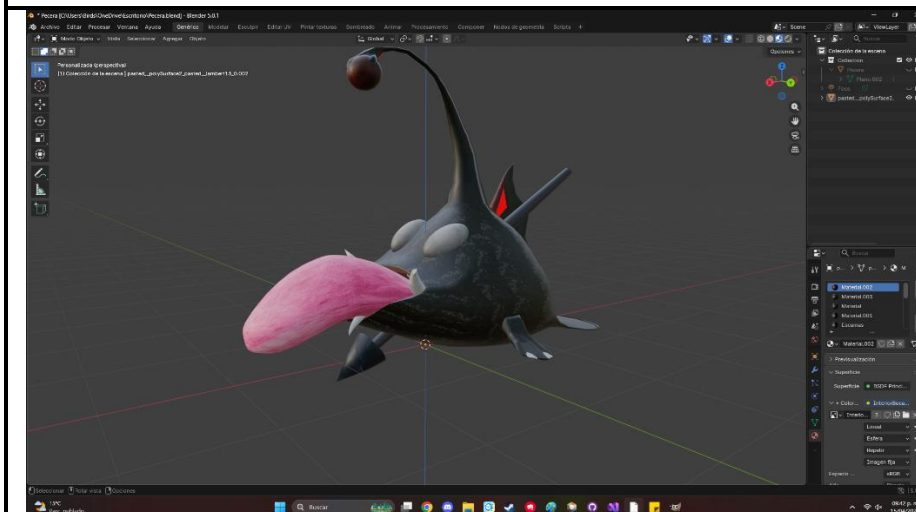
Se utilizó la sección de modificadores en blender y se buscó la opción de **SOLIDIFICAR**, ya que no es la primera vez que trabajamos con el software, sabíamos que así le daríamos grosor a nuestra pecera. ¿Con que fin? Para que tuviera una doble pared.



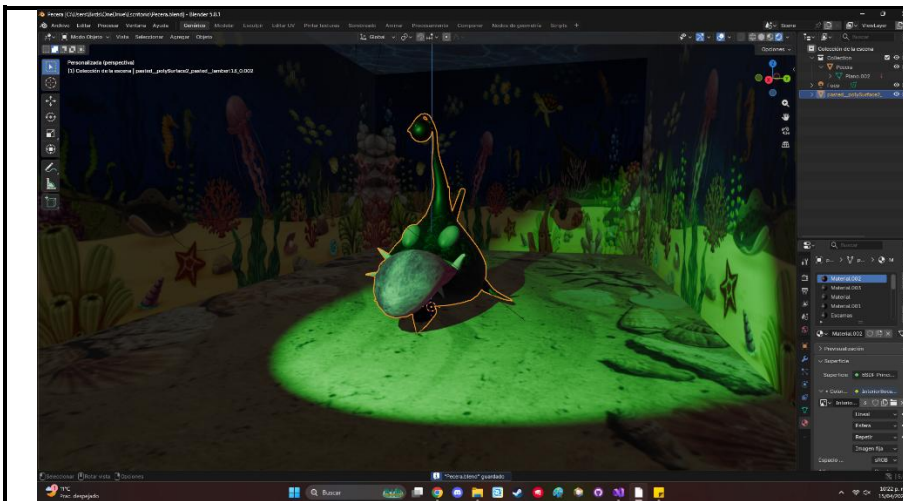
Se verificaron las normales, la doble pared funcionó para tener normales apuntando hacia dentro y hacia fuera de la pecera, simulando un mejor resultado apreciable gracias a las herramientas del modo edición y rayos X se apreció que las normales ya estaban correctas.



Se agrego una luz para entender el comportamiento resultante, aunque también se investigó que en caras planas se puede modificar blender para mostrar un color rojo o azul para saber si una normal apunta hacia dentro o fuera ***ESTO QUEDO DEMOSTRADO EN EL GIF AL FINAL DE LA TABLA**



Se importo y texturizo el pez.



Se hizo la prueba del resultado de ambos modelos en blender

GIF DE PRUEBA EN BLENDER:

<https://drive.google.com/file/d/1OCKnFRfPtxi7mrrF9HWvMR8GW1593qrE/view?usp=sharing>

Aquí podemos apreciar por un breve momento la cara roja que indica que la normal esta invertida y apuntando hacia dentro.

Pasando a los cambios en código

Window.h

Se agregaron los getters para el movimiento del pez, es importante mencionar que no solo se agrego uno de movimiento, sino también uno que nos apoyaría en la rotación para que no pareciera que nuestro pez va en reversa.

```
// --- VARIABLES PARA EL PEZ ---
GLfloat getMovPez() { return movPez; }
GLfloat getRotPez() { return rotPez; }
// --- VARIABLES PARA EL PEZ ---
GLfloat movPez;
GLfloat rotPez;
```

Window.cpp

En maneja teclado establecimos que con la tecla O incrementamos el movimiento, pero con un limite para no sobrepasar la pecera, además de girar el modelo 180° para que “mire” hacia donde se mueve. Con P, restamos pasos y regresamos a una rotación de 0°

```
// --- PEZ ABISAL ---
// [O y P]: Movimiento diagonal (Z y Y combinados) con volteo direccional
if (key == GLFW_KEY_O) {
    theWindow->movPez += 0.05f; // Avanza hacia el fondo y arriba
    if (theWindow->movPez > 3.0f) theWindow->movPez = 3.0f; // Límite superior
    theWindow->rotPez = 180.0f; // El modelo voltea hacia la dirección de avance
}
if (key == GLFW_KEY_P) {
```



```

theWindow->movPez -= 0.05f; // Retrocede hacia el frente y abajo
if (theWindow->movPez < -0.5f) theWindow->movPez = -0.5f; // Límite inferior
theWindow->rotPez = 0.0f; // El modelo voltea para no nadar de reversa
}

```

Main

Se hizo una serie de 4 pasos:

- Primero cargamos el modelo del pez y la pecera ya trabajados en blender, esto antes del while
- Luego hicimos un calculo de la diagonal que el pez va a recorrer, ya que no tenemos movimiento solo por un eje, vamos a modificar la altura y la profundidad, obteniendo un resultado de prueba y error de una multiplicación en Z de 3.5 (ya que recordemos que la pecera es rectangular y necesita avanzar más nuestro pez en Z que en Y)
- Renderizamos nuestro pez.
- Renderizamos nuestra pecera, PERO, en la ultima sección de nuestro código donde tenemos los elementos con transparencia activada.

```

Model Pecera;
Model PezAbisal;
Pecera = Model(); Pecera.LoadModel("Models/Pecera.obj");
PezAbisal = Model(); PezAbisal.LoadModel("Models/PezAbisal.obj");
...
...
// -----
// JERARQUÍA LUZ AZUL DEL PEZ ABISAL (INDICE 1 en pointLights)
// -----
// 1. Calculamos la posición del pez AQUÍ ARRIBA (Para que la luz y el modelo la usen)
posX_Pez = 0.0f;
posY_Pez = -1.0f + mainWindow.getMovPez();
posZ_Pez = -15.55f - (3.5f * mainWindow.getMovPez());
...
...
// -----
// RENDERIZADO DEL PEZ ABISAL (Trayectoria Diagonal en Y-Z)
// -----
model = glm::mat4(1.0); // Reiniciamos la matriz a la identidad
// Solo usamos las variables que ya calculamos en la Fase 1
model = glm::translate(model, glm::vec3(posX_Pez, posY_Pez, posZ_Pez));
// ROTACIÓN BASE DEL PEZ
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
// ROTACIÓN DINÁMICA (EL PEZ DE VOLTEA)
model = glm::rotate(model, mainWindow.getRotPez() * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
// ENVÍO AL SHADER Y DIBUJADO
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
PezAbisal.RenderModel();
...
...
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
// PECERA -----
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -1.95f, -20.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Pecera.RenderModel();

```

3.- Luces puntuales, encendidos y apagados.

Instrucciones: Agregar una luz de tipo puntual de color azul ligada al bulbo del pez que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara de

su práctica 7 como esta luz (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual), de tal forma que se pueda ver: las 2 luces apagadas, las 2 luces prendidas, una luz prendida y una luz apagada y viceversa.

Window.h

Al igual que el getter que teníamos para la lampara Hextech, se agrego uno para la luz puntual. Es importante mencionar que se consideró trabajar con un solo getter y pasar a través de mis cuadro combinaciones de luces (00,01,10,11) pero esta idea se descartó, ya que tener una tecla para cada luz podría ser más práctico.

```
bool getApagarLampara() { return apagarLampara; }
bool getApagarLuzPez() { return apagarLuzPez; }
```

Window.cpp

Cambiamos el cambio de colores en el faro del coche por la letra I y colocamos nuestros controles para las luces puntuales con la tecla 9 y 0 que actuarán como switches.

```
// =====
// 3. CONTROL DE ILUMINACIÓN Y EFECTOS
// =====

// [9]: Interruptor de la Lámpara Hextech (PointLight)
if (key == GLFW_KEY_9 && action == GLFW_PRESS) {
    theWindow->apagarLampara = !theWindow->apagarLampara;
}

// [0]: Interruptor de la Luz del Pez Abisal (PointLight Azul)
if (key == GLFW_KEY_0 && action == GLFW_PRESS) {
    theWindow->apagarLuzPez = !theWindow->apagarLuzPez;
}

// [I]: Ciclar colores del faro del coche (Arreglo de SpotLights)
if (key == GLFW_KEY_I && action == GLFW_PRESS) {
    theWindow->colorFaroCoche++;
    if (theWindow->colorFaroCoche > 5) { // De Magenta (5) reinicia a Rojo (0)
        theWindow->colorFaroCoche = 0;
    }
}
```

Main

Se hizo una serie de 3 pasos:

- Primero instanciamos una luz azul en nuestro arreglo de luces puntuales, se hizo de esta manera y no se recurrió al arreglo auxiliar porque solo tenemos dos luces puntuales. Además, con ayuda de un marcador (un cubo ya comentado en código) encontramos la posición a la que esta luz puntual debía estar respecto al centro de nuestro pez, mencionamos de igual manera que fue respecto al centro porque tenemos una rotación del mismo al moverlo a través de la pecera.
- Agregamos jerarquía respecto a la luz, esto lo logramos multiplicando la matriz de transformaciones del pez que incluye su traslación en diagonal y su rotación con la posición de la luz puntual, dando por resultado la posición global de la luz del abisal.
- Finalmente, aplicamos la lógica de apagado, en cada ciclo de reloj se verifica, si una luz debe estar encendida la guardamos en la primera casilla vacía de nuestro

```

arreglo lucesPuntualesActivas.
// LUZ DEL PEZ ABISAL (Índice 1)
pointLights[1] = PointLight(0.0f, 0.0f, 1.0f,          // Color Azul Puro (RGB)
                           1.0f, 1.0f,              // Intensidad
                           0.0f, 0.0f, 0.0f,          // Posición (Se sobrescribe en el while)
                           1.0f, 0.7f, 1.8f);         // Atenuación
pointLightCount++;
...
glm::vec4 posLocalLuzPez = glm::vec4(0.0f, 2.65f, 0.65f, 1.0f);
glm::vec4 posGlobalLuzPez;
...
// -----
// JERARQUÍA LUZ AZUL DEL PEZ ABISAL (INDICE 1 en pointLights)
// -----
// 1. Calculamos la posición del pez AQUÍ ARRIBA (Para que la luz y el modelo la usen)
posX_Pez = 0.0f;
posY_Pez = -1.0f + mainWindow.getMovPez();
posZ_Pez = -15.55f - (3.5f * mainWindow.getMovPez());
// 2. Creamos la matriz de la luz y le aplicamos las MISMAS transformaciones del pez
matrizPezLuz = glm::mat4(1.0);
matrizPezLuz = glm::translate(matrizPezLuz, glm::vec3(posX_Pez, posY_Pez, posZ_Pez));
matrizPezLuz = glm::rotate(matrizPezLuz, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
matrizPezLuz = glm::rotate(matrizPezLuz, mainWindow.getRotPez() * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
// 3. Multiplicamos la matriz por tu vector local
posGlobalLuzPez = matrizPezLuz * posLocalLuzPez;
// 4. LUZ DEL PEZ SIEMPRE ENCENDIDA (Azul Puro)
pointLights[1] = PointLight(0.0f, 0.0f, 1.0f, 1.0f, 1.0f, posGlobalLuzPez.x, posGlobalLuzPez.y, posGlobalLuzPez.z, 1.0f, 0.09f, 0.032f);
...
// =====
// FASE 2: ENVÍO DE LUCES AL SHADER (Empaquetado dinámico)
// =====
shaderList[0].SetDirectionalLight(&mainLight);
// Reiniciamos el contador de luces puntuales en CADA FRAME
unsigned int lucesPuntualesActivas = 0;
// --- 1. LÁMPARA HEXTECH (Tecla 9) ---
// Si NO está apagada (es decir, está prendida)
if (mainWindow.getApagarLampara() == false) {
    // La metemos en la posición actual del contador (sería la 0)
    pointLights[lucesPuntualesActivas] = PointLight(1.0f, 1.0f, 1.0f, 1.0f, 1.0f, 5.75f, -
7.0f, 1.0f, 0.09f, 0.032f);
    lucesPuntualesActivas++; // Sumamos 1 al contador
}
// --- 2. LUZ DEL PEZ ABISAL (Tecla 0) ---
// Si NO está apagada (es decir, está prendida)
if (mainWindow.getApagarLuzPez() == false) {
    // La metemos en la posición actual del contador (sería la 0 si la lámpara se apagó, o la 1 si la lámpara
está prendida)
    pointLights[lucesPuntualesActivas] = PointLight(0.0f, 0.0f, 1.0f, 1.0f, 1.0f, posGlobalLuz-
Pez.x, posGlobalLuzPez.y, posGlobalLuzPez.z, 1.0f, 0.09f, 0.032f);
    lucesPuntualesActivas++; // Sumamos 1 al contador
}
// --- ENVÍO FINAL AL SHADER ---
// Le mandamos el arreglo y nuestro contador exacto de luces activas
// Si ambas están apagadas, lucesPuntualesActivas valdrá 0, y el shader NO procesará nada.
shaderList[0].SetPointLights(pointLights, lucesPuntualesActivas);
// Las luces spot se quedan igual
shaderList[0].SetSpotLights(spotLights, spotLightCount);

```

4.- Spotlight pez abisal

Instrucciones: Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del bulbo del pez y que ilumine en cualquier dirección dentro de la pecera al presionar teclas asignadas al eje x, eje y y eje z.

Window.h

Para una rotación total se agrega rotación en cada eje de nuestra luz spot.

```
// --- VARIABLES PARA EL SPOTLIGHT DEL PEZ ---
GLfloat getRotSpotX() { return rotSpotX; }
GLfloat getRotSpotY() { return rotSpotY; }
GLfloat getRotSpotZ() { return rotSpotZ; }
```

Window.cpp

Con las teclas T,Y,U hacemos posible estas rotaciones sin limites al poder girar los 360°

```
// =====
// 3. CONTROL DE ILUMINACIÓN Y EFECTOS
// =====

// [T, Y, U]: Rotación independiente del Spotlight Amarillo del Pez
if (key == GLFW_KEY_T) {
    theWindow->rotSpotX += 5.0f; // Rotación en Eje X
}
if (key == GLFW_KEY_Y) {
    theWindow->rotSpotY += 5.0f; // Rotación en Eje Y
}
if (key == GLFW_KEY_U) {
    theWindow->rotSpotZ += 5.0f; // Rotación en Eje Z
}
```

Main

Se hizo una serie de 2 pasos principales:

- En nuestro arreglo de spotlights declaramos una luz más amarilla.
- Para colocar la luz en el pez abisal construimos la matriz de direcciones de nuestra luz spot, definiendo que al iniciar estaría apuntando hacia abajo (0,-1,0). Este fue uno de los pasos más complicados porque para que nuestra luz acompañe los giros del pez cuando da la vuelta en la pecera, convertimos las matrices a 3x3 (glm::mat3) para ignorar la traslación y quedarnos solo con la rotación. Nos valimos del método SetFlash (que actualiza posición y dirección a la vez). Le pasamos la posición posGlobalLuzPez (que calculamos en la actividad 3 para el foco azul) asegurando que salgan del mismo exacto punto, y la dirección que acabamos de calcular.

```
// Índice 2: Spotlight del Pez Abisal (Amarillo)
spotLights[2] = SpotLight(1.0f, 1.0f, 0.0f, // Color Amarillo (RGB)
    1.0f, 1.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, -1.0f, 0.0f,
    1.0f, 0.045f, 0.0075f,
    40.0f);
spotLightCount++;
...
...
...

// =====
// JERARQUÍA LUZ AMARILLA DEL PEZ (INDICE 2 en spotLights)
// =====

// 1. Calculamos las rotaciones independientes dictadas por las teclas T, Y, U
matrizDirSpot = glm::mat4(1.0);
matrizDirSpot = glm::rotate(matrizDirSpot, mainWindow.getRotSpotX() * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
matrizDirSpot = glm::rotate(matrizDirSpot, mainWindow.getRotSpotY() * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
matrizDirSpot = glm::rotate(matrizDirSpot, mainWindow.getRotSpotZ() * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
// 2. Vector base: hacia dónde apuntaría el foco si no presionáramos nada (Ej. hacia abajo)
dirBaseSpot = glm::vec3(0.0f, -1.0f, 0.0f);
// 3. IMPORTANTE: Multiplicamos la rotación propia del pez (matrizPezLuz)
// por la rotación de las teclas (matrizDirSpot) y aplicamos al vector base.
```



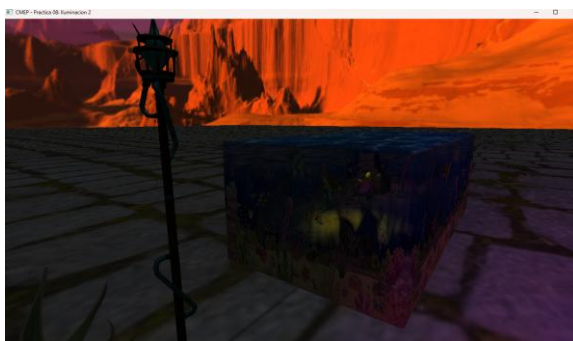
```
// (Usamos glm::mat3 para solo tomar en cuenta la rotación y descartar la traslación de la matriz).
dirFinalSpot = glm::mat3(matrizPezLuz) * glm::mat3(matrizDirSpot) * dirBaseSpot;
// 4. Actualizamos la luz: Le pasamos la misma posición que la PointLight azul (posGlobalLuzPez)
// y la nueva dirección calculada.
spotLights[2].SetFlash(glm::vec3(posGlobalLuzPez), dirFinalSpot);
```

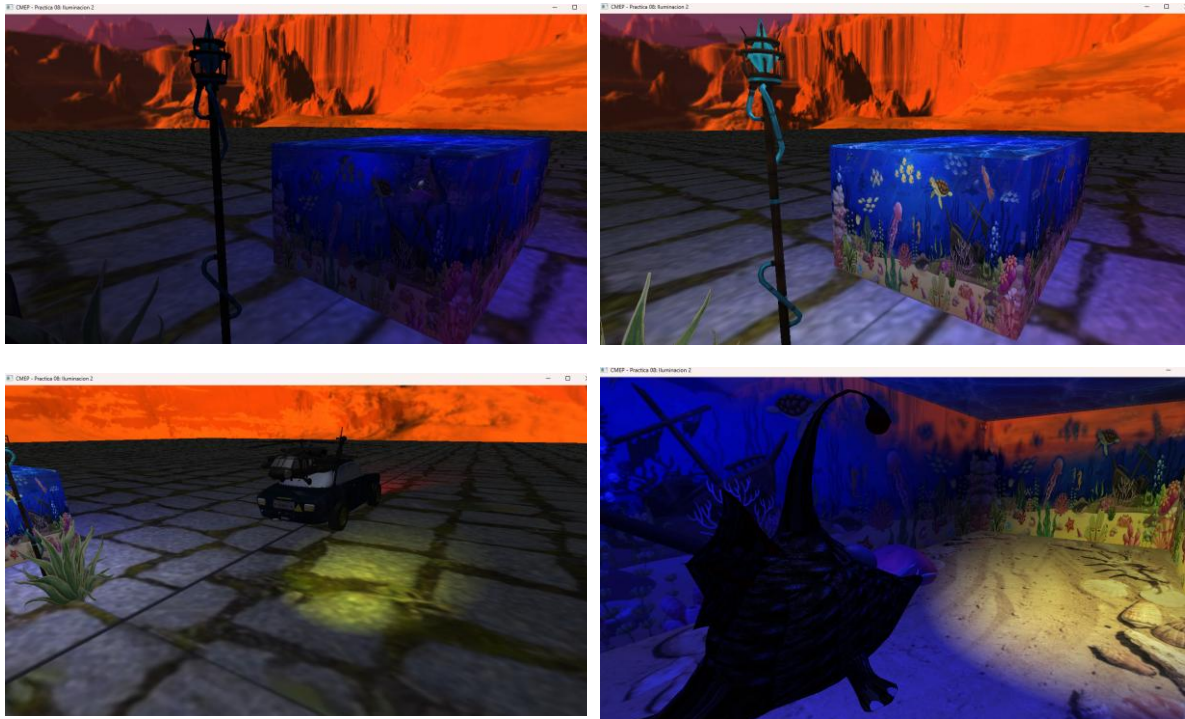
EJECUCION

Tecla	Objeto	Acción Principal
ESC	Sistema	Cierra la ventana del programa.
1 / 2	Coche	Abre (1) / Cierra (2) el cofre.
3 / 4	Coche	Avanza (4) / Retrocede (3) y gira las llantas.
5 / 6	Helicóptero	Avanza (5) / Retrocede (6) y cambia el faro direccional.
7 / 8	Dado/Octaedro	Rotación libre sobre su eje Y.
O / P	Pez Abisal	Nada en diagonal (O = fondo/arriba, P = frente/abajo) y voltea.
9	Lámpara Hextech	Enciende / Apaga la luz puntual blanca.
0	Pez Abisal	Enciende / Apaga el bulbo puntual azul.
I	Coche	Cambia de color el faro delantero (SpotLight).
T / Y / U	Foco del Pez	Rota el Spotlight amarillo en los ejes X (T), Y (Y), Z (U).

SE ADJUNTA LINK AL GIF EN DRIVE: <https://drive.google.com/file/d/1-aZ64gSla0vw0QcZs0fOxeA3eCpQUUJ2/view?usp=sharing>

IMPORTANTE: Ya que el GIF es más pesado para que Google drive lo visualice se recomienda descargarlo y observarlo desde su propio ordenador.





PROBLEMAS PRESENTADOS

Rotación y trayectoria del Pez Abisal: Al programar el movimiento en diagonal del pez dentro de la pecera, el modelo se trasladaba correctamente, pero al retroceder daba la impresión visual de estar "nadando en reversa". Esto se solucionó de manera sencilla implementando una variable maestra adicional (rotPez). Con un par de condicionales en el teclado, se le aplicó un giro de 180 grados a la matriz del modelo cuando avanzaba, y se regresaba a 0 grados al retroceder, sin embargo esto se sintió que conllevó algo más de dificultad para el jerarquizado de luces.

Empaquetado dinámico de Luces Puntuales: El requerimiento de poder apagar y encender de manera independiente la lámpara y la luz del pez para observar todas las combinaciones posibles representó un reto. En lugar de intentar restar valores al contador de luces o enviar luces de color negro (lo cual se hizo de hecho erróneamente al comenzar la práctica), se optó por reconstruir el arreglo de luces en cada ciclo de reloj. Con condicionales simples, solo se añadían al arreglo aquellas luces cuyas variables booleanas indicaban que debían estar encendidas.

CONCLUSIONES

a. Los ejercicios del reporte: Complejidad y Explicación La práctica fue bastante pesada a diferencia de las anteriores y aunque se aprendió bastante y se dejaron en claro más conceptos, se percibió que los ejercicios requerían mucho análisis previo y poca modificación en el código que hasta incluso llegó a ser repetitiva. Los primeros ejercicios, como el cambio de spotlights direccionales en el helicóptero, sirvieron para afianzar el uso de variables de estado ligadas al teclado. Sin embargo, el pico de complejidad se encontró al final de la práctica. Anidar el spotlight direccional al bulbo del pez requirió un entendimiento profundo del orden de multiplicación de matrices; tuvimos que convertir matrices de transformación a glm::mat3 para aislar las rotaciones independientes del teclado y multiplicarlas por la rotación base del pez, logrando una jerarquía algo más compleja a la que hemos trabajado, pero aparentemente más funcional.

b. Comentarios generales A nivel general, el desarrollo de la práctica fue fluido. Como sugerencia para futuras sesiones de laboratorio, considero que sería de gran ayuda dedicar un poco más de tiempo a repasar visualmente el orden matemático en el que se deben multiplicar las matrices (traslación, rotación, escalado) cuando tenemos múltiples objetos jerarquizados, o al menos otorgar más tiempo para prácticas con más números de actividades.

c. Conclusión Final El desarrollo de esta práctica consolidó significativamente nuestros conocimientos sobre la iluminación en OpenGL y el manejo de arreglos para los shaders. Además, se logró un avance importante en mi comprensión geométrica; finalmente logré entender el concepto y la importancia de las normales de manera profunda al poder visualizarlas directamente en la interfaz de Blender a través de los colores rojo y azul que indican la orientación de las caras. Ver este concepto aplicado gráficamente fue mucho más claro e intuitivo que simplemente imaginarlo como un dato de coordenadas en una matriz como se había pedido con el octaedro, lo cual me ayudó a configurar de manera exitosa las luces atrapadas dentro del grosor de la pecera de “vidrio”.

BIBLIOGRAFÍA Y MESOGRAFÍA

- Blender Foundation. (s.f.). Normals. Blender 4.1 Reference Manual. Recuperado el 15 de abril de 2026, de <https://docs.blender.org/manual/en/latest/modeling/meshes/editing/mesh/normals.html>
- Blender Foundation. (s.f.). Solidify Modifier. Blender 4.1 Reference Manual. Recuperado el 15 de abril de 2026, de <https://docs.blender.org/manual/en/latest/modeling/modifiers/generate/solidify.html>
- Blender Foundation. (s.f.). Viewport Overlays. Blender 4.1 Reference Manual. Recuperado el 15 de abril de 2026, de <https://docs.blender.org/manual/en/latest/editors/3dview/display/overlays.html>

- De Vries, J. (s.f.). Light casters. LearnOpenGL. Recuperado el 15 de abril de 2026, de <https://learnopengl.com/Lighting/Light-casters>

Mesografía (Recursos Gráficos)

- Freepik. (s.f.). Composición veraniega con arena y conchas marinas [Imagen]. Recuperado el 15 de abril de 2026, de https://www.freepik.es/foto-gratis/composicion-veraniega-arena-conchas-marinas_1139228.htm
- Freepik. (s.f.). Fondo de vector de dibujos animados paisaje submarino [Imagen]. Recuperado el 15 de abril de 2026, de https://www.freepik.es/vector-premium/fondo-vector-dibujos-animados-paisaje-submarino_48572404.htm
- Freepik. (s.f.). Fondo pastel acuarela pintado a mano aquarelle manchas colores sobre papel [Imagen]. Recuperado el 15 de abril de 2026, de https://www.freepik.es/fotos-premium/fondo-pastel-acuarela-pintado-mano-aquarelle-manchas-colores-sobre-papel_13952137.htm
- Freepik. (s.f.). Pared gris texturizada [Imagen]. Recuperado el 15 de abril de 2026, de https://www.freepik.es/foto-gratis/pared-gris-texturizada_4102961.htm
- Freepik. (s.f.). Textura de lentejuelas negras cosidas en rayas [Imagen]. Recuperado el 15 de abril de 2026, de https://www.freepik.es/fotos-premium/textura-lentejuelas-negras-cosidas-rayas-concepto-costura_17725213.htm
- Freepik. (s.f.). Texturas [Catálogo de Imágenes]. Recuperado el 15 de abril de 2026, de <https://www.freepik.es/fotos-vectores-gratis/texturas>
- Freepik. (s.f.). Toma de primer plano en ángulo alto del agua cristalina de la piscina [Imagen]. Recuperado el 15 de abril de 2026, de https://www.freepik.es/foto-gratis/toma-primer-plano-angulo-alto-agua-cristalina-piscina_11301383.htm